

Trabajo Práctico 3
Programación III - COM-02
Primer cuatrimestre 2026

Profesores:

- Gabriel Carrillo
- Gonzalo Faccini

Alumnos:

- Emiliano Acosta:
- Santiago Ruiz:
- Facundo Rios:

Fecha de entrega:

Jueves 11 de junio de 2026.

Introducción

El presente trabajo práctico aborda el problema de la construcción del equipo ideal de trabajo a partir de un conjunto de empleados con distintos roles, puntajes y posibles conflictos interpersonales. El objetivo central es encontrar el equipo que maximice el puntaje promedio, cumpliendo con la cantidad requerida de integrantes por rol y sin incluir empleados que tengan conflictos entre sí.

Se buscó utilizar todos los conceptos que vimos durante la carrera y tener en cuenta el concepto de “separated presentation”, es decir, el código que se utiliza para que la interfaz esté correctamente encapsulada y separada del código de negocio que ejecuta las funciones esenciales del programa, utilizando una arquitectura de diseño de 3 capas (model-view-presenter).

A partir de nuestra elección de arquitectura, se optó por realizar una división del código en tres paquetes principales: por un lado tenemos un paquete de **view** con la interfaz de usuario que contiene las clases pertinentes a las distintas ventanas del juego, otro paquete **model** con las clases relativas a la lógica de negocio en donde se encuentra una clase que gestiona los algoritmos para el funcionamiento del core de la aplicación, y por último un paquete de **dal** en donde se ubican las clases que interviene en el guardado y recuperación de datos de los empleados y equipos generados.

Además en este trabajo entraron los conceptos e implementación de los algoritmos de Fuerza Bruta y Backtracking, la implementación de Heurísticas y la utilización de threads para garantizar el poder seguir utilizando el programa mientras este esté resolviendo algunos de los algoritmos.

View

PantallaInicio: el inicio en donde el usuario ve los roles de empleados para armar el equipo y puede indicar cuantos de cada uno necesitará en su conformación. Todos los campos deben ser completados o un aviso visual se mostrará como alerta.

PantallaEquipo: la ventana principal de resolución del programa, incluye una lista de empleados precargada y una de conflictos, además de poseer dos secciones para agregar nuevos empleados y nuevos conflictos que no se hayan cargado, además de una serie de botones para buscar una solución con diversos métodos de forma individual y una para resolverlo con todas para ejecutar una comparativa.

PantallaComparativa: ventana emergente que se abre si se ejecuta la opción para encontrar el equipo con todos los métodos a la vez, muestra tres tablas con la configuración de empleados resultante de cada algoritmo, otra con datos comparativos de cada algoritmo ejecutado y un gráfico jFreeChart para destacar visualmente la exactitud y diferencia entre los ratings promedios resultantes de la solución de cada algoritmo.

Lógica de Negocio

Aquí tenemos una clase principal que se ocupa de la entrada y gestión de los elementos de esta capa llamada GestorEquipo y un sistema de notificación a Observers gracias a los cuales se informa la generación de un nuevo equipo a la visual y a la capa de datos. Además una clase abstracta Algoritmo de la cual heredan FuerzaBruta, Backtracking y Heurística los cuales utilizan métodos de recursión para ejecutar su proceso de búsqueda.

En el paquete **dal** ubicamos lo pertinente al almacenamiento y recuperación de los empleados con sus datos y los conflictos entre ellos y los equipos generados. Las listas de empleados y sus conflictos se encuentran en dos archivos .json, y hay dos clases de carga y guardado de datos que implementan interfaces para desacoplar y hacer nuestro código reutilizable. La clase saveData es notificada mediante un Observer y guarda en un worklog los equipos generados junto a sus datos y si se ejecutó una comparativa, crea un nuevo archivo .txt con los equipos generados por cada algoritmo y los datos de cada uno.

Conclusión

El trabajo práctico permitió analizar en profundidad el impacto que tiene la estrategia algorítmica elegida sobre la eficiencia computacional, manteniendo constante la calidad del resultado en los casos de Fuerza Bruta y Backtracking. La comparativa integrada en la aplicación hace tangible esta diferencia: Backtracking resuelve el mismo problema en menos de un milisegundo con apenas 21.800 combinaciones exploradas, mientras que Fuerza Bruta requiere más de 1.000 millones de evaluaciones y decenas de segundos.

La Heurística demuestra que en contextos donde la velocidad es crítica, sacrificar la garantía de optimalidad puede ser una decisión razonable: produce un equipo de buena calidad de forma prácticamente instantánea, con una única pasada sobre la lista de candidatos.